

Homework 8

Exercise 1

3D data set fitting with linear least squares

Feature 1

Polynomials in two variables are functions $f : \mathbb{R}^n \rightarrow \mathbb{R}$ consisting of the sum of terms of the form $bx_1^n x_2^m$, where $b \in \mathbb{R}$ is called a coefficient. The degree of each term $bx_1^n x_2^m$ is the sum of the exponents $n + m$, and the degree of the polynomials is the largest degree among all the terms of the polynomial. Examples:

$$\begin{aligned} x_1^2 x_2 - 6x_1^3 x_2^{12} + 10x_1^2 - 7x_2 + 1 & \text{ degree } 15 \\ x_1^3 x_2^2 - x_1^3 x_2^3 - x_1 x_2 + x_2^4 & \text{ degree } 6 \end{aligned} \quad (1)$$

The aim of this exercise is to implement a 3D data set fitting method using a polynomial regression by minimizing the linear least squares

$$\arg \min_{\beta \in \mathbb{R}^p} \|y - X\beta\|^2 \quad (2)$$

where the vector β contains the coefficients of the polynomial.

1. Determine p in Equation 2 in function of the degree of the polynomial.
2. Write a Python function that, given a degree, returns the corresponding matrix X
3. Using the matrix X constructed in 2, compute the solution of the minimization problem for polynomials of degree 1, 3, 4, 5, 6, 7. For each degree, plot the surface generated by the polynomial, plot the dataset and the true function as well, to show the polynomial adjusts to the data and the true function.
4. Compute the mean square error of each polynomial with respect to both the dataset and the true function. Which polynomial best approximates the true function?

Solution.

1. Given that we are only considering polynomials of degree 2, we only need to check for terms $x_1^n x_2^m$ in which $n + m = k$ for each $k = 0, 1, \dots, d$. That is,

$$\begin{aligned} \text{deg } 0 : x_1^0 x_2^0 & \rightarrow 1 \\ \text{deg } 1 : x_1^1, x_2^1 & \rightarrow 2 \\ \text{deg } 2 : x_1^2, x_1^1 x_2^1, x_2^2 & \rightarrow 3 \\ & \vdots \\ \text{deg } d : x_1^d, x_1^{d-1} x_2^1, \dots, x_2^d & \rightarrow d + 1 \end{aligned} \quad (3)$$

Thus, we have

$$p = \sum_{i=1}^{d+1} i = \frac{(d+1)(d+2)}{2}. \quad (4)$$

2. Using part 1, we build the matrix

$$X = \begin{pmatrix} 1 & x_1 & x_1^2 & \dots & x_1^p \\ 1 & x_2 & x_2^2 & \dots & x_2^p \\ \vdots & \vdots & \vdots & \dots & \vdots \\ 1 & x_n & x_n^2 & \dots & x_n^p \end{pmatrix} \quad (5)$$

That is,

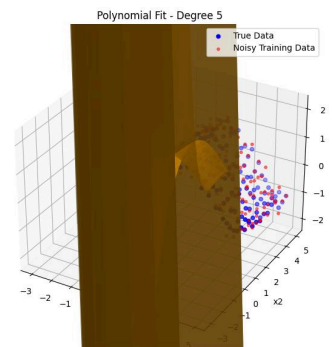
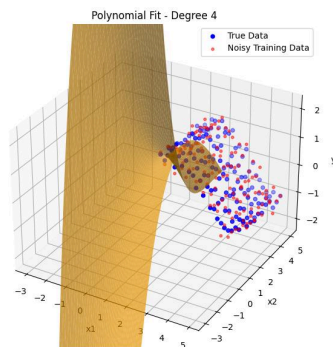
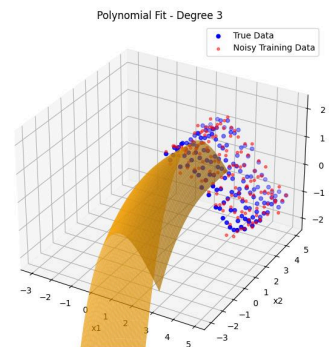
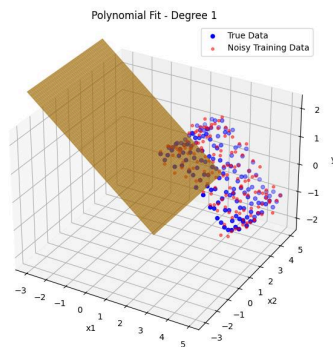
$$X = \begin{pmatrix} 1 & x_1 & x_1^2 & \dots & x_1^p \\ 1 & x_2 & x_2^2 & \dots & x_2^p \end{pmatrix} \quad (6)$$

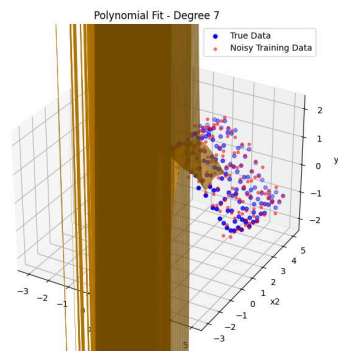
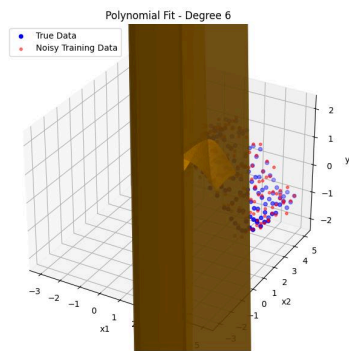
where $p = \frac{(d+1)(d+2)}{2}$. See `p1.py` for the implementation.

3. Running python `p1.py` will plot the corresponding functions.
4. Running python `p1.py` and comparing all the MSE's with the true function, we find that the polynomial of degree 3 is the best fit.

Degree	MSE (Dataset / Noisy)	MSE (True Function)
1	0.10986	0.06151
3	0.05866	0.02206
4	0.05797	0.02238
5	0.05580	0.02293
6	0.05383	0.02313
7	0.04908	0.02391

The polynomial that best approximates the true function is Degree 3.





Exercise 2

3D data set fitting with nonlinear least squares

Feature 2

Given the dataset $\{(x_{1i}, x_{2i}, y_i)\}$ of Exercise 1, and the aim of this exercise is to determine the coefficients $\beta = (\beta_0, \beta_1, \beta_2, \beta_3)^\top$ of the model

$$f(\beta; x) = \beta_0 + \beta_1 \sin(\beta_2 x_1) \cos(\beta_3 x_2) \quad (7)$$

solving the nonlinear least squares problem

$$\arg \min_{\beta} \sum_{i=1}^m r_i(\beta)^2 \quad (8)$$

where $r_{i(\beta)} = f(\beta; x_{1i}, x_{2i}) - y_i$ are the components of the residual vector

$$r(\beta) = \begin{pmatrix} r_1(\beta) \\ r_2(\beta) \\ \vdots \\ r_m(\beta) \end{pmatrix} \quad (9)$$

1. write a python function that evaluates the model $f(\beta; x)$.
2. write a python function that evaluates the residual $r(\beta)$ for the dataset of exercise 1.
3. Write a python function that computes the value of the Jacobian of $r(\beta)$, where the partial derivatives have been determined in an analytic manner.
4. Write a python function that implements the Levenberg-Marquardt algorithm to solve the non-linear least squares problem.
5. Test the algorithm with the following parameters:
 - Initial point $\beta_0 = (1, 1, 0.75, 0.5)^\top$
 - $\mu_{\text{ref}} = 0.001$
 - Threshold $\tau = \sqrt{m} \varepsilon_m^{\frac{1}{3}}$, where ε_m is the machine epsilon.
 - Maximum number of iterations $N = 200$.
 - i. Print the final vector β_k , the final value $f(x_k)$, the final number of iterations k and the variable res that indicates whether the threshold has been reached before the end of the iterations.
 - ii. Calculate the mean square error with respect to the dataset and the true function, and print the values.
 - iii. Plot the surface generated by the model, plot the dataset and the true function as well, to show how the model adjusts to the data and the true function
6. Repeat 5. with the following initial point $\beta_0 = (1, 0.5, 0.75, 0.5)^\top$.

Soltuion. Running python p2.py gives us the results of all the queries.

```
=====
Running Experiment 5
```

```
Initial beta0: [1.  1.  0.75 0.5 ]
=====
```

```
res = 1
```

```
Iterations completed: 29
```

```
Final fitted parameters (beta): [1.63090241e-02 1.00502138e+00 1.00353350e+00
4.45938984e-06]
```

```
Final f(xk): 1.5624609656622717
```

```
MSE (Model vs Dataset):      0.021701
```

```
MSE (True Function vs Dataset): 0.021891
=====
```

```
Running Experiment 6
```

```
Initial beta0: [1.  0.5  0.75 0.5 ]
=====
```

```
res = 1
```

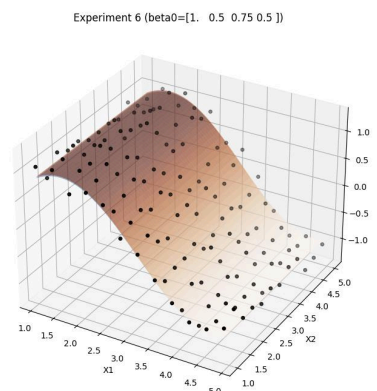
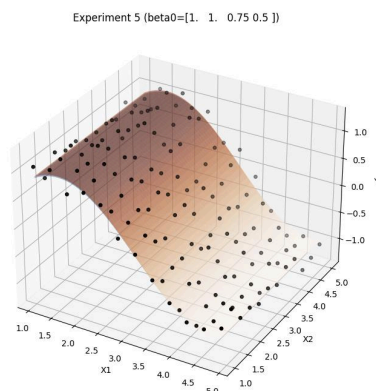
```
Iterations completed: 31
```

```
Final fitted parameters (beta): [ 1.63090230e-02 1.00502138e+00 1.00353350e+00
-3.02084351e-06]
```

```
Final f(xk): 1.5624609656525705
```

```
MSE (Model vs Dataset):      0.021701
```

```
MSE (True Function vs Dataset): 0.021891
```



Exercise 3

Constrained least-squares

Feature 3

Let $H \in \mathbb{R}^{p \times n}$ be a matrix of rank n , and the function $f : \mathbb{R}^n \rightarrow \mathbb{R}$ defined by

$$f(x) = \|y - Hx\|^2. \quad (10)$$

1. Let $C \in \mathbb{R}^{q \times n}$ and $b \in \mathbb{R}^q$. Show that f has a unique minimum on the set $S := \{x \in \mathbb{R}^n, Cx = b\}$.
2. Assuming that C is such that the matrix $C(H^\top H)^{-1}C^\top$ has rank q , determine the minimum of f on S .
3. Express the minimum of f on S in function of the minimum of f on $\mathbb{R}^n$.

Proof.

1. As seen before in class, f can be proven to be convex on $\mathbb{R}^n$. Taking into account that H has full column rank, f is strictly convex, thus if we prove S is convex, we are done. To prove it, let $x, y \in S$ and $\alpha \in [0, 1]$, then

$$\alpha Cx + (1 - \alpha)Cy = \alpha b + (1 - \alpha)b = b. \quad (11)$$

Thus, $\alpha Cx + (1 - \alpha)Cy \in S$, so S is convex, and thus f has a unique minimum in S .

2. Expanding f , we have

$$(y - Hx)^\top (y - Hx) = y^\top y - 2y^\top Hx + x^\top H^\top Hx \quad (12)$$

Thus, constraining to $x \in S$, we obtain the Lagrangian

$$\mathcal{L}(x, \lambda) = y^\top y - 2y^\top Hx + x^\top H^\top Hx + 2\lambda(Cx - b) \quad (13)$$

where we deliberately used 2λ for further simplifications. The gradient w.r.t. x is given by

$$\nabla_x \mathcal{L}(x, \lambda) = -2y^\top H + 2H^\top Hx + 2C^\top \lambda \quad (14)$$

Setting it equal to zero, we obtain

$$x = (H^\top H)^{-1}(y^\top H - C^\top \lambda). \quad (15)$$

Thus, substituting into the constraint equation, we have

$$\begin{aligned} C(H^\top H)^{-1}y^\top H - C(H^\top H)^{-1}C^\top \lambda &= b \\ \Leftrightarrow \lambda &= [C(H^\top H)^{-1}C^\top]^{-1}[C(H^\top H)^{-1}y^\top H - b], \end{aligned} \quad (16)$$

where we used that $C(H^\top H)^{-1}C^\top$ has full column rank to invert it. Substituting back into x , we have

$$x = (H^\top H)^{-1}y^\top H - (H^\top H)^{-1}C^\top [C(H^\top H)^{-1}C^\top]^{-1}[C(H^\top H)^{-1}y^\top H - b] \quad (17)$$

3. Now, we find the minimum of f in $\mathbb{R}^n$.

$$\begin{aligned} \nabla f(x) &\equiv 0 \Leftrightarrow -2y^\top H + 2H^\top Hx = 0 \\ &\Leftrightarrow x^* = (H^\top H)^{-1}y^\top H \end{aligned} \quad (18)$$

Thus, on Equation 17, we may instead write

$$x = x^* - (H^\top H)^{-1}C^\top [C(H^\top H)^{-1}C^\top]^{-1}[Cx^* - b]. \quad (19)$$

■

Exercise 4

Projection onto convex sets

Feature 4

Using KKT conditions, determine the projection operator onto:

1. the closed ball of center $(0, 0)$ and radius R .
2. a closed box $[a_1, b_1] \times [a_2, b_2]$, where $a_1 < b_1$ and $a_2 < b_2$.

Proof. We want to minimize the distance from a point y and the set

$$B_R = \{x \in \mathbb{R}^n : \|x\| \leq R\}. \quad (20)$$

That is,

$$\min_x \frac{1}{2} \|x - y\|^2, \quad x \in B_R. \quad (21)$$

Equivalently,

$$\min_{x \in \mathbb{R}^n} \frac{1}{2} \|x - y\|^2, \text{ constrained to } \|x\|^2 - R^2 \leq 0. \quad (22)$$

The associated Lagrangian is thus

$$\mathcal{L}(x, \mu) = \frac{1}{2} \|x - y\|^2 + \mu (\|x\|^2 - R^2). \quad (23)$$

The KKT conditions are:

$$\begin{aligned} \nabla_x \mathcal{L}(x, \mu) &= (x - y) + 2\mu x = 0, \\ \|x\|^2 &\leq R^2, \\ \mu &\geq 0, \\ \mu(\|x\|^2 - R^2) &= 0. \end{aligned} \quad (24)$$

The first condition gives

$$x = \frac{1}{1 + 2\mu} y. \quad (25)$$

We consider the inactive case of the constraint ($\mu = 0$), which gives

$$x = y, \quad (26)$$

and thus the second condition becomes

$$\|y\| \leq R, \quad (27)$$

so the projection is itself. If it is active ($\mu > 0$), the last condition implies

$$\|x\| = R, \quad (28)$$

and thus the first equation, after taking the norm, becomes

$$\|x\| = \frac{1}{1 + 2\mu} \|y\| = R \Leftrightarrow \frac{1}{1 + 2\mu} = \frac{R}{\|y\|}, \quad (29)$$

and thus

$$x = R \frac{y}{\|y\|}. \quad (30)$$

Thus, we conclude that the projection function onto the closed ball is

$$\pi_{B_R}(y) = \begin{cases} y & \text{if } \|y\| \leq R \\ R \frac{y}{\|y\|} & \text{if } \|y\| > R \end{cases} \quad (31)$$

2. Similar to the previous solution, we want to minimize

$$\min_x \frac{1}{2}(x_1 - y_1)^2 + \frac{1}{2}(x_2 - y_2)^2, \quad (x_1, x_2) \in \{(x_1, x_2) \in \mathbb{R}^2 : a_1 \leq x_1 \leq b_1, a_2 \leq x_2 \leq b_2\} \quad (32)$$

The constraints may be written as

$$\begin{aligned} x_1 - b_1 &\leq 0 \\ x_2 - b_2 &\leq 0 \\ a_1 - x_1 &\leq 0 \\ a_2 - x_2 &\leq 0 \end{aligned} \quad (33)$$

Since these inequalities are independent of each other (for each component), and we may also separate the objective function as a sum of independent functions, thus we may simplify the analysis of the Lagrangian to a component-wise analysis. That is,

$$\begin{aligned} \mathcal{L}(x_1, \lambda, \mu) &= \frac{1}{2}(x_1 - y_1)^2 + \lambda(x_1 - b_1) + \mu(x_1 - a_1) \\ \mathcal{L}(x_2, \lambda, \mu) &= \frac{1}{2}(x_2 - y_2)^2 + \lambda(x_2 - b_2) + \mu(x_2 - a_2). \end{aligned} \quad (34)$$

And therefore,

$$\nabla_{x_i} \mathcal{L}(x_i, \lambda, \mu) = (x_i - y_i) + \lambda - \mu \quad (35)$$

- If $y_i \in [a_i, b_i]$, setting $\mu = \lambda = 0$, we obtain $x_i = y_i$ and thus x_i satisfies the conditions.
- If $y_i > b_i$, assume the upper bound is active, so $x_i = b_i$, and the lower is not, so that $\mu = 0$. Equation 35 then gives $\lambda = y_i - b_i > 0$, satisfying the condition, so $x_i = b_i$.
- If $y_i < a_i$, assume the lower bound is active, so $x_i = a_i$ and the upper is inactive, so $\lambda = 0$. Equation 35 gives $\mu = a_i - y_i > 0$, satisfying the condition, so $x_i = a_i$.

Therefore, the projection operator is given by

$$\pi_{[a_1, b_1] \times [a_2, b_2]}(y_1, y_2) = \begin{cases} y_i & \text{if } y_i \in [a_i, b_i] \\ a_i & \text{if } y_i < a_i \\ b_i & \text{if } y_i > b_i \end{cases} = \begin{pmatrix} \min(\max(y_1, a_1), b_1) \\ \min(\max(y_2, a_2), b_2) \end{pmatrix} \quad (36)$$

■